National Real-Time Payments System

Architecture Solutions Report

Architecture Foundations Assignment

Case Study: Designing a Scalable, Secure, Real-Time Payment Platform

Prepared for: UNext Learning Pvt Ltd.

1. Systems and Subsystems Required

The National Real-Time Payments System is decomposed into the following major systems, each with focused subsystems to ensure separation of concerns, independent scalability, and fault isolation.

System	Key Subsystems
User Management System	KYC/Identity Verification Service, User Profile Service, Authentication & Authorization Service (OAuth 2.0 + MFA), Device Management Service
Payment Processing Engine	Transaction Orchestrator (SAGA-based), Payment Router, QR Code Payment Service, Idempotency Manager
Banking Integration Gateway	Bank Adapter Framework (per-bank adapters), Account Linking & Validation Service, Real-Time Debit/Credit Instruction Service
Settlement & Reconciliation	Net Settlement Engine (batch cycles), Reconciliation Service, Dispute & Chargeback Manager, Deferred Settlement Queue
Merchant Platform	Merchant Onboarding & Verification, Merchant Dashboard & Analytics, Daily/Weekly Settlement Reports, QR Code Management
Fraud & Risk Engine	Real-Time ML Fraud Detector (inline, <200ms), Rule-Based Risk Scoring, Behavioral Analytics, Watchlist & Sanctions Screening
Notification & Communication	Push Notification Service, SMS/Email Gateway, In-App Messaging, Incident Status Page
Admin & Compliance Platform	Admin Dashboard, Regulatory Reporting Engine, Immutable Audit Log (append-only), Data Residency & Sovereignty Manager
API Gateway & Integration Layer	API Gateway with Rate Limiting, Developer Portal & SDK, Third-Party Integration APIs, Merchant POS Integration
Infrastructure & Observability	Load Balancer & Service Mesh, Auto-Scaling Controller, Distributed Tracing & Monitoring, Alerting & Incident Management

Design Principle: Each system is deployed as an independent microservice cluster, communicating via asynchronous event streams (Apache Kafka) for non-critical paths and synchronous gRPC for the payment critical path.

2. Information Exchange Between Systems

The following table details the data flows, their characteristics, and consistency requirements. The critical payment path demands synchronous, strongly consistent communication, while supporting systems use asynchronous, eventually consistent messaging.

Source	Destination	Frequency	Speed	Volume	Consistency	Protocol
User App	Payment Engine	Very High	<500ms	50K TPS	Strong	gRPC/REST
Payment Engine	Fraud Engine	Very High	<200ms	50K TPS	Strong	gRPC (inline)
Payment Engine	Bank Gateway	Very High	<1s	50K TPS	Strong	gRPC
Bank Gateway	Partner Banks	High	Real-time	Millions/day	Strong	ISO 20022
Settlement Eng.	Banks	Scheduled	Minutes	Aggregated	Eventual	Batch/SFTP
Notification	Users	Very High	<5s	50K TPS	Best-effort	Push/SMS
All Systems	Audit Logger	Continuous	Async	Very High	Strong	Kafka
Monitoring	Admin Dash	Continuous	Near RT	High	Eventual	Prometheus

Key Design Decision: The payment critical path (User -> Payment Engine -> Fraud -> Bank -> Response) uses synchronous gRPC with strict 2-second end-to-end SLA. All non-critical paths (notifications, audit, analytics) use Kafka-based async messaging for decoupling and resilience.

3. Major Potential Failure Points

A. Technical Failures

- Database overload or corruption during peak 50,000+ TPS causing transaction loss
- Payment Engine crash with in-flight transactions stuck in intermediate state
- Message queue (Kafka) backlog causing delayed processing of settlements and notifications
- Cache (Redis) invalidation failure leading to stale account balances shown to users
- Auto-scaling lag during sudden traffic spikes (10x during national events)

B. Integration Failures

- Partner bank API timeout or complete downtime (partial bank outage)
- KYC/Identity verification provider unavailability blocking new user onboarding
- QR code generation or validation service failure disrupting merchant payments
- Third-party fraud intelligence data feed disruption reducing detection accuracy

C. External Dependency Failures

- Network partition between data centers causing split-brain scenarios
- DNS resolution failures preventing service discovery
- TLS certificate expiry causing handshake failures across services
- Regulatory/compliance system downtime blocking mandatory checks
- Cloud provider regional outage affecting hosted infrastructure

4. Failure Mitigation Strategies

A defense-in-depth approach is adopted, layering multiple mitigation strategies to ensure no single failure can cascade into a system-wide outage.

Strategy	Applied To	How It Helps
Active-Active Multi-Region	All critical services	Eliminates SPOF; enables 99.999% uptime with automatic geo-failover
Circuit Breakers	Bank Gateway, External APIs	Prevents cascade failures; fast-fails when downstream is unhealthy
Synchronous DB Replication	Transaction Database	Zero data loss guarantee across regions (RPO = 0)
Event Sourcing + WAL	Payment Engine	Recovers in-flight transactions after crash; full audit trail
Idempotency Keys	All Payment APIs	Prevents duplicate transactions on network retries
Rate Limiting & Throttling	API Gateway	Protects backend from traffic spikes, DDoS, and abuse
Bulkhead Pattern	Service Isolation	Failure in one subsystem does not propagate to others
Auto-Healing + Health Checks	All Infrastructure	Detects and replaces unhealthy instances in <30 seconds
Encryption (AES-256 + TLS 1.3)	All Data Paths	PCI DSS and ISO 27001 compliance; data protection
Pre-Scaling via Event Calendar	Compute & DB	Infrastructure scaled before predicted peak events

5. Contingency Plans for Failure Scenarios

A. Manual Reconciliation

When automated settlement fails, a reconciliation job compares internal transaction logs against bank ledger statements. A dedicated reconciliation team uses a specialized dashboard to identify and resolve mismatches within a T+1 SLA. All discrepancies are logged for audit.

B. Deferred Settlements

If a partner bank is unreachable, settlement instructions are queued in a durable Kafka topic with configurable retention (default: 7 days). Once the bank recovers, queued settlements are automatically replayed in order. Merchants see a 'Pending Settlement' status on their dashboard with estimated resolution time.

C. User Communication Strategy

- Delayed Transaction: Immediate push notification - 'Your payment is being processed. We will notify you once complete.'
- Failed Transaction: Clear notification with next steps - 'Payment failed. Amount will be refunded to your account within 24 hours.'
- System-Wide Incident: Public status page (similar to UPI's approach) with real-time updates and estimated resolution time.
- Proactive Alerts: SMS/email for high-value transaction failures with dedicated support contact.

D. Operational Runbooks

Pre-defined runbooks for every P1/P2 failure scenario. On-call rotation with escalation matrix. War-room protocol activated when failures affect >1% of transactions. Post-incident reviews within 48 hours with published RCA (Root Cause Analysis).

6. System Behavior Under Different Loads

A. Normal Daily Load (~20,000 TPS)

- Standard auto-scaling configuration with baseline instance count
- Full ML-based fraud detection on every transaction (complete model inference)
- Synchronous per-transaction bank communication
- All features fully enabled: transaction history, receipts, dashboards, analytics
- Standard database connection pool sizing

B. Peak National Events (up to 10x = 200,000+ TPS)

- Pre-scaled infrastructure triggered 2-4 hours before predicted peak via event calendar
- Simplified fraud path: rule-based fast-track for low-risk transactions (<$50), full ML only for flagged/high-value
- Micro-batch settlements every 5 minutes instead of per-transaction settlement
- Graceful degradation: transaction history queries throttled, dashboard refresh intervals increased to 5 min
- Read replicas scaled 3x for query-heavy operations
- CDN and edge caching activated for static content (QR templates, merchant assets)
- Non-essential background jobs (analytics, reporting) paused to free compute resources

Key Principle: The payment critical path is NEVER degraded. Only supporting features are throttled or simplified during peak load to protect core transaction processing.

7. Architectural Trade-offs

A. Consistency vs. Availability

Decision: Strong consistency on the payment critical path (account debit/credit), eventual consistency for non-critical reads (transaction history, dashboards, analytics).

Reasoning: Financial transactions cannot tolerate inconsistency - a user must never see an incorrect balance. The SAGA pattern is used for distributed transactions, ensuring eventual correctness with compensating actions. Per the CAP theorem, during network partitions, the payment path chooses consistency (CP), while read-heavy services choose availability (AP).

B. Cost vs. Reliability

Decision: Premium active-active multi-region infrastructure for the payment engine and transaction database. Cost-optimized eventual-consistency stores for analytics, logs, and reporting.

Reasoning: 99.999% uptime means maximum 5.26 minutes downtime per year. The cost of downtime (regulatory fines, reputational damage, user trust erosion) far exceeds the infrastructure premium. Non-critical systems use cheaper single-region deployments with async replication.

C. Real-Time vs. Eventual Processing

Decision: Real-time processing for transaction execution (<2s end-to-end). Eventual/batch processing for inter-bank settlement (every 15-30 minutes), regulatory reporting, and analytics.

Reasoning: Users expect instant payment confirmation. However, inter-bank net settlement does not need to be instant - batch settlement is industry standard (used by UPI, PIX, FedNow) and significantly reduces bank integration complexity and cost.

D. Security Depth vs. User Experience

Decision: Risk-based adaptive authentication. Low-value transactions require PIN only. High-value or unusual transactions require biometric + OTP. Suspicious transactions trigger step-up authentication.

Reasoning: Requiring full MFA for every small payment would severely impact adoption rates. The fraud engine's real-time risk score dynamically determines the authentication level, balancing security with frictionless user experience.

8. Global References & Recommended USPs

Reference Implementations

System	Country	Key Strengths
UPI	India	10B+ monthly txns, interoperable across banks, QR-based, open API ecosystem enabling PhonePe/GPay
PIX	Brazil	24/7 instant payments, 150M+ users in 2 years, QR + NFC, free for individuals
FedNow	USA	Federal Reserve backed, ISO 20022 messaging, focus on reliability and compliance
Faster Payments	UK	Sub-second settlement, 24/7 availability since 2008, proven at scale

Recommended Unique Selling Propositions (USPs)

Feature	Impact	Estimated ROI
Offline Payment Mode	Enables payments in low-connectivity rural areas via encrypted tokens; increases financial inclusion	+15-20% user base in underserved regions
AI Smart Routing	Auto-routes transactions via fastest/cheapest bank path; reduces failures	~30% reduction in transaction failure rate
Programmable Payments	Auto-pay, escrow, conditional payments for merchants and enterprises	New revenue streams; attracts enterprise clients
ISO 20022 from Day 1	Cross-border ready architecture; no re-engineering for international expansion	Saves 12-18 months of future re-engineering
Open Banking APIs	Developer portal enabling third-party fintech innovation on the platform	Ecosystem growth similar to UPI's success
Green Ledger	Carbon savings tracking for digital vs cash transactions; ESG compliance	Attracts sustainability-focused investors

Impact Summary: By combining proven patterns from UPI (interoperability), PIX (rapid adoption), and FedNow (reliability) with the above USPs, the platform becomes not just a national payment rail but an extensible financial infrastructure. The offline mode and smart routing directly address the two biggest pain points in emerging-market digital payments: connectivity and reliability. The ISO 20022 foundation ensures seamless international expansion when the time comes.